```python
# Step 1: Import the tools we need
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Step 2: Create a small dataset (posts + their sentiment labels)
posts = [
    "I love this app, it's awesome!",
    "This is the worst update ever.",
    "Not bad, but could be better.",
    "Amazing experience, I'm so happy!",
    "I hate using this, very annoying."
]

labels = ["positive", "negative", "neutral", "positive", "negative"]

# Step 3: Convert text into numbers (Bag of Words)
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(posts)

# Step 4: Train a simple model (Naive Bayes)
model = MultinomialNB()
model.fit(X, labels)

# Step 5: Test the model with new posts
new_posts = [
    "I really enjoy using this app!",
    "Terrible service, I'm angry.",
    "It's okay, nothing special."
]

X_new = vectorizer.transform(new_posts)
predictions = model.predict(X_new)
```

```python
# Step 6: Show results
for post, sentiment in zip(new_posts, predictions):
    print(f"Post: {post}")
    print(f"Predicted Sentiment: {sentiment}")
    print()
    """
validator.py
Checks if the user's input text is safe, meaningful and not empty.
"""

import re

def is_non_empty_text(text: str) -> bool:
    """Returns True only if the user actually typed something."""
    return isinstance(text, str) and text.strip() != ""

def is_text_reasonable_length(text: str, min_len=1, max_len=5000) -> bool:
    """Rejects extremely long or extremely short text."""
    if not isinstance(text, str):
        return False
    length = len(text.strip())
    return min_len <= length <= max_len

def contains_allowed_characters(text: str) -> bool:
    """Checks if the text contains normal letters or numbers."""
    if not isinstance(text, str):
        return False
    return bool(re.search(r"[A-Za-z0-9]", text))
"""
predictor.py
Uses the trained model to guess the sentiment of cleaned text.
```

```python
import numpy as np

def predict_sentiment(model, vectorizer, text):
    """
    Takes cleaned text → converts to numbers → model predicts → returns label.
    """

    # Convert single text to list for consistent processing
    single = False
    if isinstance(text, str):
        text = [text]
        single = True

    # Convert text to numeric form for the ML model
    vectorized = vectorizer.transform(text)

    # Predict the sentiment (positive/negative/neutral)
    prediction = model.predict(vectorized)

    # If model supports probability scores
    if hasattr(model, "predict_proba"):
        probability = model.predict_proba(vectorized)
        confidence = probability.max(axis=1)
    else:
        confidence = [1.0] * len(prediction)   # basic fallback

    results = []
    for label, score in zip(prediction, confidence):
        results.append({
            "label": label,
            "confidence": float(score)
```

```python
def predict_sentiment(model, vectorizer, text):

    return results[0] if single else results
"""
model_loader.py
Loads and saves the machine learning model and the TF-IDF vectorizer.
"""

import os
from joblib import dump, load

MODEL_DIR = "model"
os.makedirs(MODEL_DIR, exist_ok=True)

MODEL_PATH = os.path.join(MODEL_DIR, "sentiment_model.joblib")
VECTORIZER_PATH = os.path.join(MODEL_DIR, "vectorizer.joblib")

def save_model(model, vectorizer):
    """Saves model + vectorizer for future predictions."""
    dump(model, MODEL_PATH)
    dump(vectorizer, VECTORIZER_PATH)

def load_model_and_vectorizer():
    """Loads model + vectorizer. Raises an error if not trained yet."""
    if not os.path.exists(MODEL_PATH):
        raise FileNotFoundError("No model found. Run train.py first.")
    if not os.path.exists(VECTORIZER_PATH):
        raise FileNotFoundError("Vectorizer missing. Run train.py first.")

    return load(MODEL_PATH), load(VECTORIZER_PATH)
"""
train.py
```

```python
Steps:
1. Load dataset (CSV with text + label)
2. Clean text
3. Transform text into numbers (TF-IDF)
4. Train Logistic Regression
5. Evaluate performance
6. Save the model + vectorizer
"""

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, f1_score
from preprocess import clean_text
from model_loader import save_model
from logging_config import get_logger

logger = get_logger("trainer")

def train(csv_path):
    # Load dataset
    data = pd.read_csv(csv_path)

    # Clean text column
    data["cleaned"] = data["text"].astype(str).apply(clean_text)

    X = data["cleaned"]
    y = data["label"]

    # Split into train/test
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=42
```

```python
def train(csv_path):

    # Convert text to numeric features
    vectorizer = TfidfVectorizer(max_features=15000)
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    # Train the model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train_vec, y_train)

    # Test performance
    predictions = model.predict(X_test_vec)
    accuracy = accuracy_score(y_test, predictions)
    f1score = f1_score(y_test, predictions, average="weighted")

    logger.info(f"Accuracy: {accuracy:.3f}")
    logger.info(f"F1 Score: {f1score:.3f}")
    logger.info("\n" + classification_report(y_test, predictions))

    # Save model + vectorizer
    save_model(model, vectorizer)

    print("Training complete. Model saved.")
    """
main.py
This is the user-facing program.
User types text → system cleans → predicts → shows result.
"""

from model_loader import load_model_and_vectorizer
from preprocess import clean_text
from validator import (
```

```python
    is_non_empty_text,
    is_text_reasonable_length,
    contains_allowed_characters,
)
from predictor import predict_sentiment

def start():
    print("=== Sentiment Analysis System ===")
    print("Type a message to analyze. Type 'exit' to quit.\n")

    # Load trained model
    try:
        model, vectorizer = load_model_and_vectorizer()
    except FileNotFoundError:
        print("Model not found. Train the model first using train.py")
        return

    while True:
        user_input = input("Enter text: ")

        if user_input.lower() == "exit":
            print("Goodbye!")
            break

        # Validate input
        if not is_non_empty_text(user_input):
            print("⚠ Please type something.")
            continue
        if not contains_allowed_characters(user_input):
            print("⚠ Text doesn't contain valid characters.")
            continue
        if not is_text_reasonable_length(user_input):
            print("⚠ Text too long or too short.")
```

```python
def start():

        cleaned = clean_text(user_input)
        result = predict_sentiment(model, vectorizer, cleaned)

        print(f"\n📌 Sentiment: {result['label']}")
        print(f"📊 Confidence: {result['confidence']:.2f}\n")

if __name__ == "__main__":
    start()
```

```python
def start():

        cleaned = clean_text(user_input)
        result = predict_sentiment(model, vectorizer, cleaned)

        print(f"\n📌 Sentiment: {result['label']}")
        print(f"📊 Confidence: {result['confidence']:.2f}\n")

if __name__ == "__main__":
    start()
```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Post: Terrible service, I'm angry.
Predicted Sentiment: negative

Post: Terrible service, I'm angry.
Predicted Sentiment: negative
Post: Terrible service, I'm angry.
Predicted Sentiment: negative
Predicted Sentiment: negative

Post: It's okay, nothing special.
Predicted Sentiment: positive

Traceback (most recent call last):
```

Ln 28, Col 38 Spaces: 4 UTF-8 CRLF Python Signed out 3.13.5 (base) Go Live